

这个内存有一个地址输入，一个写的的数据输入，以及一个读的数据输出。同寄存器文件一样，从内存中读的操作方式类似于组合逻辑：如果我们在输入 `address` 上提供一个地址，并将 `write` 控制信号设置为 0，那么在经过一些延迟之后，存储在那个地址上的值会出现在输出 `data` 上。如果地址超出了范围，`error` 信号会设置为 1，否则就设置为 0。写内存是由时钟控制的：我们将 `address` 设置为期望的地址，将 `data in` 设置为期望的值，而 `write` 设置为 1。然后当我们控制时钟时，只要地址是合法的，就会更新内存中指定的位置。对于读操作来说，如果地址是不合法的，`error` 信号会被设置为 1。这个信号是由组合逻辑产生的，因为所需要的边界检查纯粹就是地址输入的函数，不涉及保存任何状态。

旁注 现实的存储器设计

真实微处理器中的存储器系统比我们在设计中假想的这个简单的存储器要复杂得多。它是由几种形式的硬件存储器组成的，包括几种随机访问存储器和磁盘，以及管理这些设备的各种硬件和软件机制。存储器系统的设计和特点在第 6 章中描述。

不过，我们简单的存储器设计可以用于较小的系统，它提供了更复杂系统的处理器和存储器之间接口的抽象。

我们的处理器还包括另外一个只读存储器，用来读指令。在大多数实际系统中，这两个存储器被合并为一个具有双端口的存储器：一个用来读指令，另一个用来读或者写数据。

4.3 Y86-64 的顺序实现

现在已经有了实现 Y86-64 处理器所需要的部件。首先，我们描述一个称为 SEQ(“sequential”顺序的)的处理器。每个时钟周期上，SEQ 执行处理一条完整指令所需的所有步骤。不过，这需要一个很长的时钟周期时间，因此时钟周期频率会低到不可接受。我们开发 SEQ 的目标就是提供实现最终目的的第一步，我们的最终目的是实现一个高效的、流水线化的处理器。

4.3.1 将处理组织成阶段

通常，处理一条指令包括很多操作。将它们组织成某个特殊的阶段序列，即使指令的动作差异很大，但所有的指令都遵循统一的序列。每一步的具体处理取决于正在执行的指令。创建这样一个框架，我们就能够设计一个充分利用硬件的处理器。下面是关于各个阶段以及各阶段内执行操作的简略描述：

- **取指(fetch)**：取指阶段从内存读取指令字节，地址为程序计数器(PC)的值。从指令中抽取出指令指示符字节的两个四位部分，称为 `icode`(指令代码)和 `ifun`(指令功能)。它可能取出一个寄存器指示符字节，指明一个或两个寄存器操作数指示符 `rA` 和 `rB`。它还可能取出一个四字节常数字 `valC`。它按顺序方式计算当前指令的下一条指令的地址 `valP`。也就是说，`valP` 等于 PC 的值加上已取出指令的长度。
- **译码(decode)**：译码阶段从寄存器文件读入最多两个操作数，得到值 `valA` 和/或 `valB`。通常，它读入指令 `rA` 和 `rB` 字段指明的寄存器，不过有些指令是读寄存器 `%rsp` 的。
- **执行(execute)**：在执行阶段，算术/逻辑单元(ALU)要么执行指令指明的操作(根据 `ifun` 的值)，计算内存引用的有效地址，要么增加或减少栈指针。得到的值我们称为 `valE`。在此，也可能设置条件码。对一条条件传送指令来说，这个阶段会检验条件码和传送条件(由 `ifun` 给出)，如果条件成立，则更新目标寄存器。同样，

对一条跳转指令来说,这个阶段会决定是不是应该选择分支。

- **访存(memory)**: 访存阶段可以将数据写入内存,或者从内存读出数据。读出的值为 valM。
- **写回(write back)**: 写回阶段最多可以写两个结果到寄存器文件。
- **更新 PC(PC update)**: 将 PC 设置成下一条指令的地址。

处理器无限循环,执行这些阶段。在我们简化的实现中,发生任何异常时,处理器就会停止:它执行 halt 指令或非法指令,或它试图读或者写非法地址。在更完整的设计中,处理器会进入异常处理模式,开始执行由异常的类型决定的特殊代码。

从前面的讲述可以看出,执行一条指令是需要进行很多处理的。我们不仅必须执行指令所表明的操作,还必须计算地址、更新栈指针,以及确定下一条指令的地址。幸好每条指令的整个流程都比较相似。因为我们想使硬件数量尽可能少,并且最终将它映射到一个二维的集成电路芯片的表面,在设计硬件时,一个非常简单而一致的结构是非常重要的。降低复杂度的一种方法是让不同的指令共享尽量多的硬件。例如,我们的每个处理器设计都只含有一个算术/逻辑单元,根据所执行的指令类型的不同,它的使用方式也不同。在硬件上复制逻辑块的成本比软件中有重复代码的成本大得多。而且在硬件系统中处理许多特殊情况和特性要比用软件来处理困难得多。

我们面临的一个挑战是将每条不同指令所需要的计算放入到上述那个通用框架中。我们会使用图 4-17 中所示的代码来描述不同 Y86-64 指令的处理。图 4-18~图 4-21 中的表描述了不同 Y86-64 指令在各个阶段是怎样处理的。很值得仔细研究一下这些表。表中的这种格式很容易映射到硬件。表中的每一行都描述了一个信号或存储状态的分配(用分配操作—来表示)。阅读时可以把它看成是从上至下的顺序求值。当我们将这些计算映射到硬件时,会发现其实并不需要严格按照顺序来执行这些求值。

1	0x000: 30f20900000000000000		irmovq \$9, %rdx	
2	0x00a: 30f31500000000000000		irmovq \$21, %rbx	
3	0x014: 6123		subq %rdx, %rbx	# subtract
4	0x016: 30f48000000000000000		irmovq \$128,%rsp	# Problem 4.13
5	0x020: 40436400000000000000		rmmovq %rsp, 100(%rbx)	# store
6	0x02a: a02f		pushq %rdx	# push
7	0x02c: b00f		popq %rax	# Problem 4.14
8	0x02e: 73400000000000000000		je done	# Not taken
9	0x037: 80410000000000000000		call proc	# Problem 4.18
10	0x040:		done:	
11	0x040: 00		halt	
12	0x041:		proc:	
13	0x041: 90		ret	# Return
14				

图 4-17 Y86-64 指令序列示例。我们会跟踪这些指令通过各个阶段的处理

图 4-18 给出了对 OPq(整数和逻辑运算)、rmmovq(寄存器-寄存器传送)和 irmovq(立即数-寄存器传送)类型的指令所需的处理。让我们先来考虑一下整数操作。回顾图 4-2, 可以看到我们小心地选择了指令编码, 这样四个整数操作(addq、subq、andq 和 xorq)都有相同的 icode 值。我们可以以相同的步骤顺序来处理它们, 除了 ALU 计算必须根据 ifun 中编码的具体的指令操作来设定。

阶段	OPq rA, rB	rrmovq rA, rB	irmovqV, rB
取指	icode; ifun $\leftarrow M_1[PC]$ rA: rB $\leftarrow M_1[PC+1]$ valP $\leftarrow PC+2$	icode; ifun $\leftarrow M_1[PC]$ rA: rB $\leftarrow M_1[PC+1]$ valP $\leftarrow PC+2$	icode; ifun $\leftarrow M_1[PC]$ rA: rB $\leftarrow M_1[PC+1]$ valC $\leftarrow M_8[PC+2]$ valP $\leftarrow PC+10$
译码	valA $\leftarrow R[rA]$ valB $\leftarrow R[rB]$	valA $\leftarrow R[rA]$	
执行	valE $\leftarrow valB \text{ OP } valA$ Set CC	valE $\leftarrow 0 + valA$	valE $\leftarrow 0 + valC$
访存			
写回	R[rB] $\leftarrow valE$	R[rB] $\leftarrow valE$	R[rB] $\leftarrow valE$
更新 PC	PC $\leftarrow valP$	PC $\leftarrow valP$	PC $\leftarrow valP$

图 4-18 Y86-64 指令 OPq、rrmovq 和 irmovq 在顺序实现中的计算。这些指令计算了一个值，并将结果存放在寄存器中。符号 icode; ifun 表明指令字节的两个组成部分，而 rA: rB 表明寄存器指示字节节的两个组成部分。符号 $M_1[x]$ 表示访问(读或者写)内存位置 x 处的一个字节，而 $M_8[x]$ 表示访问八个字节

整数操作指令的处理遵循上面列出的通用模式。在取指阶段，我们不需要常数字，所以 valP 就计算为 $PC+2$ 。在译码阶段，我们要读两个操作数。在执行阶段，它们和功能指示符 ifun 一起再提供给 ALU，这样一来 valE 就成为了指令结果。这个计算是用表达式 $valB \text{ OP } valA$ 来表达的，这里 OP 代表 ifun 指定的操作。要注意两个参数的顺序——这个顺序与 Y86-64(和 x86-64)的习惯是一致的。例如，指令 `subq %rax, %rdx` 计算的是 $R[\%rdx] - R[\%rax]$ 的值。这些指令在访存阶段什么也不做，而在写回阶段，valE 被写入寄存器 rB，然后 PC 设为 valP，整个指令的执行就结束了。

旁注 跟踪 subq 指令的执行

作为一个例子，让我们来看看一条 subq 指令的处理过程，这条指令是图 4-17 所示目标代码的第 3 行中的 subq 指令。可以看到前面两条指令分别将寄存器 %rdx 和 %rbx 初始化成 9 和 21。我们还能看到指令位于地址 0x014，由两个字节组成，值分别为 0x61 和 0x23。这条指令处理的各个阶段如下表所示，左边列出了处理一个 OPq 指令的通用的规则(图 4-18)，而右边列出的是对这条具体指令的计算。

阶段	OPq rA, rB	subq %rdx, %rbx
取指	icode; ifun $\leftarrow M_1[PC]$ rA: rB $\leftarrow M_1[PC+1]$ valP $\leftarrow PC+2$	icode; ifun $\leftarrow M_1[0x014] = 6:1$ rA: rB $\leftarrow M_1[0x015] = 2:3$ valP $\leftarrow 0x014 + 2 = 0x016$
译码	valA $\leftarrow R[rA]$ valB $\leftarrow R[rB]$	valA $\leftarrow R[\%rdx] = 9$ valB $\leftarrow R[\%rbx] = 21$
执行	valE $\leftarrow valB \text{ OP } valA$ Set CC	valE $\leftarrow 21 - 9 = 12$ ZF $\leftarrow 0$, SF $\leftarrow 0$, OF $\leftarrow 0$
访存		
写回	R[rB] $\leftarrow valE$	R[\%rbx] $\leftarrow valE = 12$
更新 PC	PC $\leftarrow valP$	PC $\leftarrow valP = 0x016$

这个跟踪表明我们达到了理想的效果，寄存器%rbx设成了12，三个条件码都设成了0，而PC加了2。

执行 `rrmovq` 指令和执行算术运算类似。不过，不需要取第二个寄存器操作数。我们将ALU的第二个输入设为0，先把它和第一个操作数相加，得到 $valE = valA$ ，然后再把这个值写到寄存器文件。对 `irmovq` 的处理与此类似，除了ALU的第一个输入为常数值 $valC$ 。另外，因为是长指令格式，对于 `irmovq`，程序计数器必须加10。所有这些指令都不改变条件码。

 **练习题 4.13** 填写下表的右边一栏，这个表描述的是图 4-17 中目标代码第 4 行上的 `irmovq` 指令的处理情况：

阶段	通用	具体
	<code>irmovq V, rB</code>	<code>irmovq \$128, %rsp</code>
取指	$icode:ifun \leftarrow M_1[PC]$ $rA:rB \leftarrow M_1[PC+1]$ $valC \leftarrow M_8[PC+2]$ $valP \leftarrow PC+10$	
译码		
执行	$valE \leftarrow 0 + valC$	
访存		
写回	$R[rB] \leftarrow valE$	
更新 PC	$PC \leftarrow valP$	

这条指令的执行会怎样改变寄存器和PC呢？

图 4-19 给出了内存读写指令 `rmmovq` 和 `mrmmovq` 所需要的处理。基本流程也和前面的一样，不过是用ALU来加 $valC$ 和 $valB$ ，得到内存操作的有效地址（偏移量与基址寄存器值之和）。在访存阶段，会将寄存器值 $valA$ 写到内存，或者从内存中读出 $valM$ 。

阶段	<code>rmmovq rA, D(rB)</code>	<code>mrmmovq D(rB), rA</code>
取指	$icode:ifun \leftarrow M_1[PC]$ $rA:rB \leftarrow M_1[PC+1]$ $valC \leftarrow M_8[PC+2]$ $valP \leftarrow PC+10$	$icode:ifun \leftarrow M_1[PC]$ $rA:rB \leftarrow M_1[PC+1]$ $valC \leftarrow M_8[PC+2]$ $valP \leftarrow PC+10$
译码	$valA \leftarrow R[rA]$ $valB \leftarrow R[rB]$	$valB \leftarrow R[rB]$
执行	$valE \leftarrow valB + valC$	$valE \leftarrow valB + valC$
访存	$M_8[valE] \leftarrow valA$	$valE \leftarrow M_8[valE]$
写回		
		$R[rA] \leftarrow valM$
更新 PC	$PC \leftarrow valP$	$PC \leftarrow valP$

图 4-19 Y86-64 指令 `rmmovq` 和 `mrmmovq` 在顺序实现中的计算。这些指令读或者写内存

旁注 跟踪 `rmmovq` 指令的执行

让我们来看看图 4-17 中目标代码的第 5 行 `rmmovq` 指令的处理情况。可以看到，前面的指令已将寄存器%rsp初始化成了128，而%rbx仍然是 `subq` 指令（第 3 行）算出来的

结果 12。我们还可以看到，指令位于地址 0x020，有 10 个字节。前两个的值为 0x40 和 0x43，后 8 个是数字 0x0000000000000064(十进制数 100)按字节反过来得到的数。各个阶段的处理如下：

阶段	通用	具体
	$\text{rmmovq } rA, D(rB)$	$\text{rmmovq } \%rsp, 100(\%rbx)$
取指	$\text{icode; ifun} \leftarrow M_1[PC]$ $rA; rB \leftarrow M_1[PC+1]$ $\text{valP} \leftarrow M_8[PC+2]$ $\text{valP} \leftarrow PC+10$	$\text{icode; ifun} \leftarrow M_1[0x020]=4:0$ $rA; rB \leftarrow M_1[0x021]=4:3$ $\text{valC} \leftarrow M_8[0x022]=100$ $\text{valP} \leftarrow 0x020+10=0x02a$
译码	$\text{valA} \leftarrow R[rA]$ $\text{valB} \leftarrow R[rB]$	$\text{valA} \leftarrow R[\%rsp]=128$ $\text{valB} \leftarrow R[\%rbx]=12$
执行	$\text{valE} \leftarrow \text{valB} + \text{valC}$	$\text{valE} \leftarrow 12+100=112$
访存	$M_8[\text{valE}] \leftarrow \text{valA}$	$M_8[112] \leftarrow 128$
写回		
更新 PC	$PC \leftarrow \text{valP}$	$PC \leftarrow 0x02a$

跟踪记录表明这条指令的效果就是将 128 写入内存地址 112，并将 PC 加 10。

图 4-20 给出了处理 pushq 和 popq 指令所需的步骤。它们可以算是最难实现的 Y86-64 指令了，因为它们既涉及访问内存，又要增加或减少栈指针。虽然这两条指令的流程比较相似，但是它们还是有很重要的区别。

阶段	pushq rA	popq rA
取指	$\text{icode; ifun} \leftarrow M_1[PC]$ $rA; rB \leftarrow M_1[PC+1]$ $\text{valP} \leftarrow PC+2$	$\text{icode; ifun} \leftarrow M_1[PC]$ $rA; rB \leftarrow M_1[PC+1]$ $\text{valP} \leftarrow PC+2$
译码	$\text{valA} \leftarrow R[rA]$ $\text{valB} \leftarrow R[\%rsp]$	$\text{valA} \leftarrow R[\%rsp]$ $\text{valB} \leftarrow R[\%rsp]$
执行	$\text{valE} \leftarrow \text{valB} + (-8)$	$\text{valE} \leftarrow \text{valB} + 8$
访存	$M_8[\text{valE}] \leftarrow \text{valA}$	$\text{valE} \leftarrow M_8[\text{valA}]$
写回	$R[\%rsp] \leftarrow \text{valE}$	$R[\%rsp] \leftarrow \text{valE}$ $R[rA] \leftarrow \text{valM}$
更新 PC	$PC \leftarrow \text{valP}$	$PC \leftarrow \text{valP}$

图 4-20 Y86-64 指令 pushq 和 popq 在顺序实现中的计算。这些指令将值压入或弹出栈

pushq 指令开始时很像我们前面讲过的指令，但是在译码阶段，用 %rsp 作为第二个寄存器操作数的标识符，将栈指针赋值为 valB。在执行阶段，用 ALU 将栈指针减 8。减过 8 的值就是内存写的地址，在写回阶段还会存回到 %rsp 中。将 valE 作为写操作的地址，是遵循 Y86-64(和 x86-64)的惯例，也就是在写之前，pushq 应该先将栈指针减去 8，即使栈指针的更新实际上是在内存操作完成之后才进行的。

旁注 跟踪 pushq 指令的执行

让我们来看看图 4-17 中目标代码的第 6 行 pushq 指令的处理情况。此时，寄存器 %rdx 的值为 9，而寄存器 %rsp 的值为 128。我们还可以看到指令是位于地址 0x02a，有两个字节，值分别为 0xa0 和 0x2f。各个阶段的处理如下：

阶段	通用	具体
	pushq rA	pushq %rdx
取指	icode:ifun $\leftarrow M_1[PC]$ rA:rB $\leftarrow M_1[PC+1]$ valP $\leftarrow PC+2$	icode:ifun $\leftarrow M_1[0x02a]=a:0$ rA:rB $\leftarrow M_1[0x02b]=2:f$ valP $\leftarrow 0x02a+2=0x02c$
译码	valA $\leftarrow R[rA]$ valB $\leftarrow R[\%rsp]$	valA $\leftarrow R[\%rdx]=9$ valB $\leftarrow R[\%rsp]=128$
执行	valE $\leftarrow valB+(-8)$	valE $\leftarrow 128+(-8)=120$
访存	$M_8[valE] \leftarrow valA$	$M_8[120] \leftarrow 9$
写回	$R[\%rsp] \leftarrow valE$	$R[\%rsp] \leftarrow 120$
更新 PC	$PC \leftarrow valP$	$PC \leftarrow 0x02c$

跟踪记录表明这条指令的效果就是将`%rsp`设为120，将9写入地址120，并将PC加2。

`popq`指令的执行与`pushq`的执行类似，除了在译码阶段要读两次栈指针以外。这样做看上去很多余，但是我们会看到让`valA`和`valB`都存放栈指针的值，会使后面的流程跟其他的指令更相似，增强设计的整体一致性。在执行阶段，用ALU给栈指针加8，但是用没加过8的原始值作为内存操作的地址。在写回阶段，要用加过8的栈指针更新栈指针寄存器，还要将寄存器`rA`更新为从内存中读出的值。用没加过8的值作为内存读地址，保持了Y86-64(和x86-64)的惯例，`popq`应该首先读内存，然后再增加栈指针。

 **练习题 4.14** 填写下表的右边一栏，这个表描述的是图4-17中目标代码第7行`popq`指令的处理情况：

阶段	通用	具体
	popq rA	popq %rax
取指	icode:ifun $\leftarrow M_1[PC]$ rA:rB $\leftarrow M_1[PC+1]$ valP $\leftarrow PC+2$	
译码	valA $\leftarrow R[\%rsp]$ valB $\leftarrow R[\%rsp]$	
执行	valE $\leftarrow valB+8$	
访存	valM $\leftarrow M_8[valA]$	
写回	$R[\%rsp] \leftarrow valE$ $R[rA] \leftarrow valM$	
更新 PC	$PC \leftarrow valP$	

这条指令的执行会怎样改变寄存器和PC呢？

 **练习题 4.15** 根据图4-20中列出的步骤，指令`pushq %rsp`会有什么样的效果？这与练习题4.7中确定的Y86-64期望的行为一致吗？

 **练习题 4.16** 假设`popq`在写回阶段中的两个寄存器写操作按照图4-20列出的顺序进行。`popq %rsp`执行的效果会是怎样的？这与练习题4.8中确定的Y86-64期望的行为一致吗？

图 4-21 表明了三类控制转移指令的处理：各种跳转、call 和 ret。可以看到，我们能用同前面指令一样的整体流程来实现这些指令。

阶段	jXX Dest	call Dest	ret
取指	$\text{icode; ifun} \leftarrow M_1[\text{PC}]$ $\text{valC} \leftarrow M_8[\text{PC}+1]$ $\text{valP} \leftarrow \text{PC}+9$	$\text{icode; ifun} \leftarrow M_1[\text{PC}]$ $\text{valC} \leftarrow M_8[\text{PC}+1]$ $\text{valP} \leftarrow \text{PC}+9$	$\text{icode; ifun} \leftarrow M_1[\text{PC}]$ $\text{valP} \leftarrow \text{PC}+1$
译码		$\text{valB} \leftarrow R[\%rsp]$	$\text{valA} \leftarrow R[\%rsp]$ $\text{valB} \leftarrow R[\%rsp]$
执行	$\text{Cnd} \leftarrow \text{Cond}(\text{CC}, \text{ifun})$	$\text{valE} \leftarrow \text{valB} + (-8)$	$\text{valE} \leftarrow \text{valB} + 8$
访存		$M_8[\text{valE}] \leftarrow \text{valP}$	$\text{valM} \leftarrow M_8[\text{valA}]$
写回		$R[\%rsp] \leftarrow \text{valE}$	$R[\%rsp] \leftarrow \text{valE}$
更新 PC	$\text{PC} \leftarrow \text{Cnd?valC:valP}$	$\text{PC} \leftarrow \text{valC}$	$\text{PC} \leftarrow \text{valM}$

图 4-21 Y86-64 指令 jXX、call 和 ret 在顺序实现中的计算。这些指令导致控制转移

同对整数操作一样，我们能够以一种统一的方式处理所有的跳转指令，因为它们的不同只在于判断是否要选择分支的时候。除了不需要一个寄存器指示符字节以外，跳转指令在取指和译码阶段都和前面讲的其他指令类似。在执行阶段，检查条件码和跳转条件来确定是否要选择分支，产生出一个一位信号 Cnd。在更新 PC 阶段，检查这个标志，如果这个标志为 1，就将 PC 设为 valC(跳转目标)，如果为 0，就设为 valP(下一条指令的地址)。我们的表示法 $x?a:b$ 类似于 C 语句中的条件表达式——当 x 非零时，它等于 a ，当 x 为零时，等于 b 。

旁注 跟踪 je 指令的执行

让我们来看看图 4-17 中目标代码的第 8 行 je 指令的处理情况。subq 指令(第 3 行)已经所有的条件码都置为了 0，所以不会选择分支。该指令位于地址 0x02e，有 9 个字节。第一个字节的值为 0x73，而剩下的 8 个字节是数字 0x0000000000000040 按字节反过来得到的数，也就是跳转的目标。各个阶段的处理如下：

阶段	通用	具体
	jXX Dest	je 0x040
取指	$\text{icode; ifun} \leftarrow M_1[\text{PC}]$ $\text{valC} \leftarrow M_8[\text{PC}+1]$ $\text{valP} \leftarrow \text{PC}+9$	$\text{icode; ifun} \leftarrow M_1[0x02e]=7:3$ $\text{valC} \leftarrow M_8[0x02f]=0x040$ $\text{valP} \leftarrow 0x02e+9=0x037$
译码		
执行	$\text{Cnd} \leftarrow \text{Cond}(\text{CC}, \text{ifun})$	$\text{Cnd} \leftarrow \text{Cond}(\langle 0, 0, 0 \rangle, 3)=0$
访存		
写回		
更新 PC	$\text{PC} \leftarrow \text{Cnd?valC:valP}$	$\text{PC} \leftarrow 0? 0x040:0x037=0x037$

就像这个跟踪记录表明的那样，这条指令的效果就是将 PC 加 9。

 **练习题 4.17** 从指令编码(图 4-2 和图 4-3)我们可以看出，rrmovq 指令是一类更通用的、包括条件转移在内的指令的无条件版本。请给出你要如何修改下面 rrmovq 指令

的步骤，使之也能处理 6 个条件传送指令。看看 jXX 指令的实现(图 4-21)是如何处理条件行为的，可能会有所帮助。

阶段	cmovXX rA, rB
取指	$icode: ifun \leftarrow M_1[PC]$ $rA: rB \leftarrow M_1[PC+1]$ $valP \leftarrow PC+2$
译码	$valA \leftarrow R[rA]$
执行	$valE \leftarrow 0 + valA$
访存	
写回	$R[rB] \leftarrow valE$
更新 PC	$PC \leftarrow valP$

指令 call 和 ret 与指令 pushq 和 popq 类似，除了我们要将程序计数器的值入栈和出栈以外。对指令 call，我们要将 valP，也就是 call 指令后紧跟着的那条指令的地址，压入栈中。在更新 PC 阶段，将 PC 设为 valC，也就是调用的目的地。对指令 ret，在更新 PC 阶段，我们将 valM，即从栈中取出的值，赋值给 PC。

 **练习题 4.18** 填写下表的右边一栏，这个表描述的是图 4-17 中目标代码第 9 行 call 指令的处理情况：

阶段	通用	具体
	call Dest	call 0x041
取指	$icode: ifun \leftarrow M_1[PC]$ $valC \leftarrow M_8[PC+1]$ $valP \leftarrow PC+9$	
译码	$valB \leftarrow R[\%rsp]$	
执行	$valE \leftarrow valB + (-8)$	
访存	$M_8[valE] \leftarrow valP$	
写回	$R[\%rsp] \leftarrow valE$	
更新 PC	$PC \leftarrow valC$	

这条指令的执行会怎样改变寄存器、PC 和内存呢？

我们创建了一个统一的框架，能处理所有不同类型的 Y86-64 指令。虽然指令的行为大不相同，但是我们可以将指令的处理组织成 6 个阶段。现在我们的任务是创建硬件设计来实现这些阶段，并把它们连接起来。

旁注 跟踪 ret 指令的执行

让我们来看看图 4-17 中目标代码的第 13 行 ret 指令的处理情况。指令的地址是 0x041，只有一个字节的编码，0x90。前面的 call 指令将 %rsp 置为了 120，并将返回地址 0x040 存放在了内存地址 120 中。各个阶段的处理如下：

阶段	通用	具体
	ret	ret
取指	$icode; ifun \leftarrow M_1[PC]$	$icode; ifun \leftarrow M_1[0x041] = 9:0$
	$valP \leftarrow PC + 1$	$valP \leftarrow 0x041 + 1 = 0x042$
译码	$valA \leftarrow R[\%rsp]$	$valA \leftarrow R[\%rsp] = 120$
	$valB \leftarrow R[\%rsp]$	$valB \leftarrow R[\%rsp] = 120$
执行	$valE \leftarrow valB + 8$	$valE \leftarrow 120 + 8 = 128$
访存	$valM \leftarrow M_8[valA]$	$valM \leftarrow M_8[120] = 0x040$
写回	$R[\%rsp] \leftarrow valE$	$R[\%rsp] \leftarrow 128$
更新 PC	$PC \leftarrow valM$	$PC \leftarrow 0x040$

跟踪记录表明这条指令的效果就是将 PC 设为 0x040，halt 指令的地址。同时也将 %rsp 置为了 128。

4.3.2 SEQ 硬件结构

实现所有 Y86-64 指令所需要的计算可以被组织成 6 个基本阶段：取指、译码、执行、访存、写回和更新 PC。图 4-22 给出了一个能执行这些计算的硬件结构的抽象表示。程序计数器放在寄存器中，在图中左下角(标明为“PC”)。然后，信息沿着线流动(多条线组合在一起就用宽一点的灰线来表示)，先向上，再向右。同各个阶段相关的硬件单元(hardware units)负责执行这些处理。在右边，反馈线路向下，包括要写到寄存器文件的更新值，以及更新的程序计数器值。正如在 4.3.3 节中讨论的那样，在 SEQ 中，所有硬件单元的处理都在一个时钟周期内完成。这张图省略了一些小的组合逻辑块，还省略了所有用来操作各个硬件单元以及将相应的值路由到这些单元的控制逻辑。稍后会补充这些细节。我们从下往上画处理器和流程的方法似乎有点奇怪。在开始设计流水线的处理器时，我们会解释这么画的原因。

硬件单元与各个处理阶段相关联：

取指：将程序计数器寄存器作为地址，指令内存读取指令的字节。PC 增加器(PC incrementer)计算 valP，即增加了的程序计数器。

译码：寄存器文件有两个读端口 A 和 B，从这两个端口同时读寄存器值 valA 和 valB。

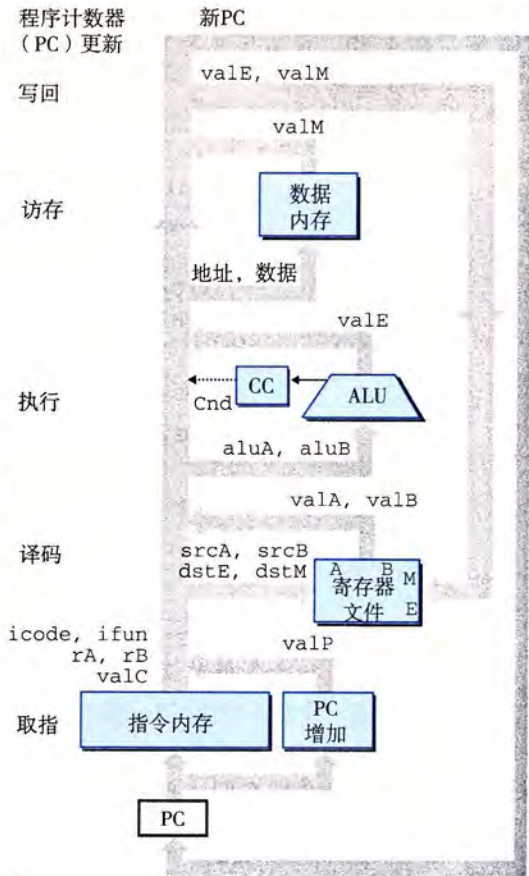


图 4-22 SEQ 的抽象视图，一种顺序实现。指令执行过程中的信息处理沿着顺时针方向的流程进行，从用程序计数器(PC)取指令开始，如图中左下角所示

执行：执行阶段会根据指令的类型，将算术/逻辑单元(ALU)用于不同的目的。对整数操作，它要执行指令所指定的运算。对其他指令，它会作为一个加法器来计算增加或减少栈指针，或者计算有效地址，或者只是简单地加0，将一个输入传递到输出。

条件码寄存器(CC)有三个条件码位。ALU 负责计算条件码的新值。当执行条件传送指令时，根据条件码和传送条件来计算决定是否更新目标寄存器。同样，当执行一条跳转指令时，会根据条件码和跳转类型来计算分支信号 Cnd。

访存：在执行访存操作时，数据内存读出或写入一个内存字。指令和数据内存访问的是相同的内存位置，但是用于不同的目的。

写回：寄存器文件有两个写端口。端口 E 用来写 ALU 计算出来的值，而端口 M 用来写从数据内存中读出的值。

PC 更新：程序计数器的新值选择自：valP，下一条指令的地址；valC，调用指令或跳转指令指定的目标地址；valM，从内存读取的返回地址。

图 4-23 更详细地给出了实现 SEQ 所需要的硬件(分析每个阶段时，我们会看到完整的

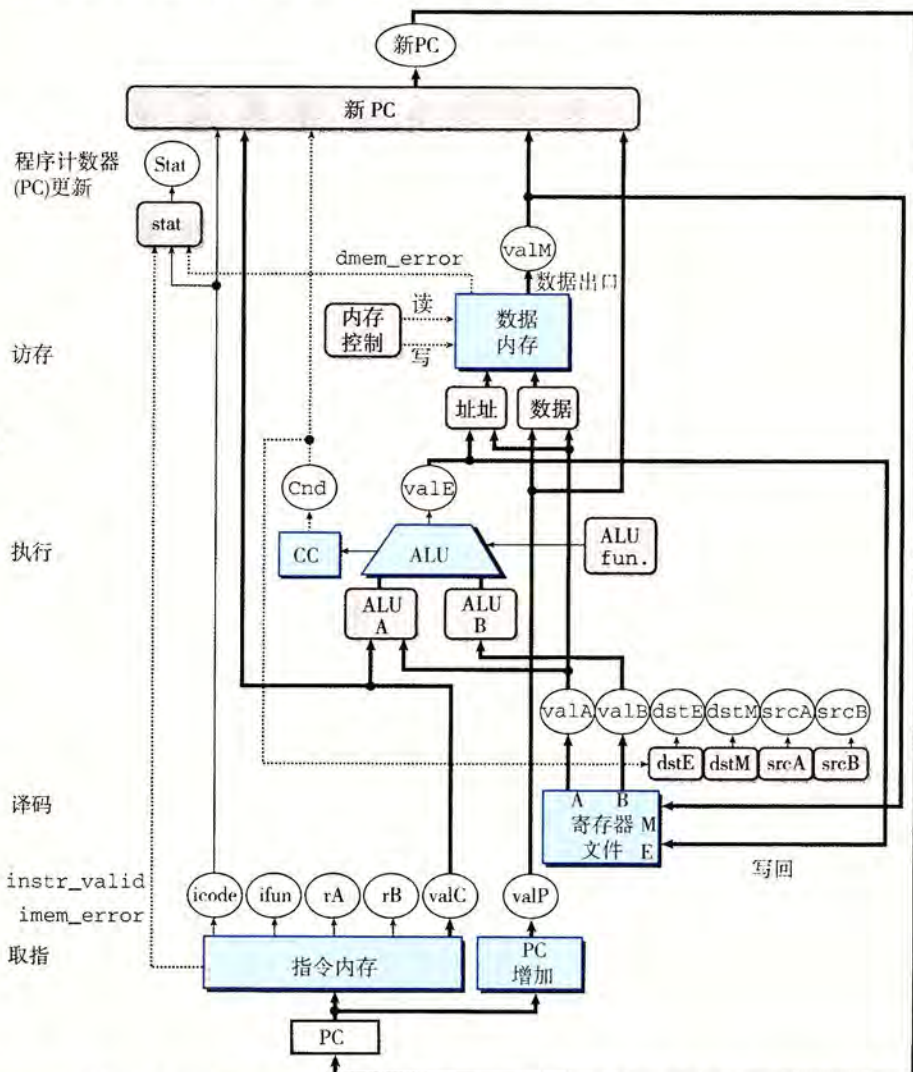


图 4-23 SEQ 的硬件结构，一种顺序实现。有些控制信号以及寄存器和控制字连接没有画出来

细节)。我们看到一组和前面一样的硬件单元，但是现在线路看得更清楚了。这幅图以及其他的硬件图都使用的是下面的画图惯例。

- 白色方框表示时钟寄存器。程序计数器 PC 是 SEQ 中唯一的时钟寄存器。
- 浅蓝色方框表示硬件单元。这包括内存、ALU 等等。在我们所有的处理器实现中，都会使用这一组基本的单元。我们把这些单元当作“黑盒子”，不关心它们的细节设计。
- 控制逻辑块用灰色圆角矩形表示。这些块用来从一组信号源中进行选择，或者用来计算一些布尔函数。我们会非常详细地分析这些块，包括给出 HCL 描述。
- 线路的名字在白色圆图中说明。它们只是线路的标识，而不是什么硬件单元。
- 宽度为字长的数据连接用中等粗度的线表示。每条这样的线实际上都代表一簇 64 根线，并列地连在一起，将一个字从硬件的一个部分传送到另一部分。
- 宽度为字节或更窄的数据连接用细线表示。根据线上要携带的值的类型，每条这样的线实际上都代表一簇 4 根或 8 根线。
- 单个位的连接用虚线来表示。这代表芯片上单元与块之间传递的控制值。

图 4-18~图 4-21 中所有的计算都有这样的性质，每一行都代表某个值的计算(如 valP)，或者激活某个硬件单元(如内存)。图 4-24 的第二栏列出了这些计算和动作。除了我们已经讲过的那些信号以外，还列出了四个寄存器 ID 信号：srcA，valA 的源；srcB，valB 的源；dstE，写入 valE 的寄存器；以及 dstM，写入 valM 的寄存器。

阶段	计算	OPq rA, rB	mrmovq D(rB), rA
取指	icode: ifun rA, rB valC valP	icode: ifun $\leftarrow M_1[PC]$ rA: rB $\leftarrow M_1[PC+1]$ valP $\leftarrow PC+2$	icode: ifun $\leftarrow M_1[PC]$ rA: rB $\leftarrow M_1[PC+1]$ valC $\leftarrow M_6[PC+2]$ valP $\leftarrow PC+10$
译码	valA, srcA valB, srcB	valA $\leftarrow R[rA]$ valB $\leftarrow R[rB]$	valB $\leftarrow R[rB]$
执行	valE Cond. codes	valE $\leftarrow valB \text{ OP } valA$ Set CC	valE $\leftarrow valB + valC$
访存	Read/write		valM $\leftarrow M_8[valE]$
写回	E port, dstE M port, dstM	R[rB] $\leftarrow valE$	R[rA] $\leftarrow valM$
更新 PC	PC	PC $\leftarrow valP$	PC $\leftarrow valP$

图 4-24 标识顺序实现中的不同计算步骤。第二栏标识出 SEQ 阶段中正在被计算的值，或正在被执行的运算。以指令 OPq 和 mrmovq 的计算作为示例

图中，右边两栏给出的是指令 OPq 和 mrmovq 的计算，来说明要计算的值。要将这些计算映射到硬件上，我们要实现控制逻辑，它能在不同硬件单元之间传送数据，以及操作这些单元，使得对每个不同的指令执行指定的运算。这就是控制逻辑块的目标，控制逻辑块在图 4-23 中用灰色圆角方框表示。我们的任务就是依次经过每个阶段，创建这些块的详细设计。

4.3.3 SEQ 的时序

在介绍图 4-18~图 4-21 的表时，我们说过要把它们看成是用程序符号写的，那些赋值是从上到下顺序执行的。然而，图 4-23 中硬件结构的操作运行根本完全不同，一个时

钟变化会引发一个经过组合逻辑的流,来执行整个指令。让我们来看看这些硬件怎样实现表中列出的这一行为。

SEQ的实现包括组合逻辑和两种存储器设备:时钟寄存器(程序计数器和条件码寄存器),随机访问存储器(寄存器文件、指令内存和数据内存)。组合逻辑不需要任何时序或控制——只要输入变化了,值就通过逻辑门网络传播。正如提到过的那样,我们也将读随机访问存储器看成和组合逻辑一样的操作,根据地址输入产生输出字。对于较小的存储器来说(例如寄存器文件),这是一个合理的假设,而对于较大的电路来说,可以用特殊的时钟电路来模拟这个效果。由于指令内存只用来读指令,因此我们可以将这个单元看成是组合逻辑。

现在还剩四个硬件单元需要对它们的时序进行明确的控制——程序计数器、条件码寄存器、数据内存和寄存器文件。这些单元通过一个时钟信号来控制,它触发将新值装载到寄存器以及将值写到随机访问存储器。每个时钟周期,程序计数器都会装载新的指令地址。只有在执行整数运算指令时,才会装载条件码寄存器。只有在执行 `rmmovq`、`pushq` 或 `call` 指令时,才会写数据内存。寄存器文件的两个写端口允许每个时钟周期更新两个程序寄存器,不过我们可以用特殊的寄存器 ID `0xF` 作为端口地址,来表明在此端口不应该执行写操作。

要控制处理器中活动的时序,只需要寄存器和内存的时钟控制。硬件获得了如图 4-18~图 4-21 的表中所示的那些赋值顺序执行一样的效果,即使所有的状态更新实际上同时发生,且只在时钟上升开始下一个周期时。之所以能保持这样的等价性,是由于 Y86-64 指令集的本质,因为我们遵循以下原则组织计算:

原则:从不回读

处理器从来不需要为了完成一条指令的执行而去读由该指令更新了的状态。

这条原则对实现的成功来说至关重要。为了说明问题,假设我们对 `pushq` 指令的实现是先将 `%rsp` 减 8,再将更新后的 `%rsp` 值作为写操作的地址。这种方法同前面所说的那个原则相违背。为了执行内存操作,它需要先从寄存器文件中读更新过的栈指针。然而,我们的实现(图 4-20)产生出减后的栈指针值,作为信号 `valE`,然后再用这个信号既作为寄存器写的地址,也作为内存写的地址。因此,在时钟上升开始下一个周期时,处理器就可以同时执行寄存器写和内存写了。

再举个例子来说明这条原则,我们可以看到有些指令(整数运算)会设置条件码,有些指令(跳转指令)会读取条件码,但没有指令必须既设置又读取条件码。虽然要有时钟上升开始下一个周期时,才会设置条件码,但是在任何指令试图读之前,它们都会更新。

以下是汇编代码,左边列出的是指令地址,图 4-25 给出了 SEQ 硬件如何处理其中第 3 和第 4 行指令:

```

1    0x000:  irmovq $0x100,%rbx    # %rbx <-- 0x100
2    0x00a:  irmovq $0x200,%rdx    # %rdx <-- 0x200
3    0x014:  addq %rdx,%rbx        # %rbx <-- 0x300 CC <-- 000
4    0x016:  je dest               # Not taken
5    0x01f:  rmmovq %rbx,0(%rdx)   # M[0x200] <-- 0x300
6    0x029:  dest: halt

```

标号为 1~4 的各个图给出了 4 个状态单元,还有组合逻辑,以及状态单元之间的连接。组合逻辑被条件码寄存器环绕着,因为有的组合逻辑(例如 ALU)产生输入到条件码寄存器,而其他部分(例如分支计算和 PC 选择逻辑)又将条件码寄存器作为输入。图中寄

寄存器文件和数据内存有独立的读连接和写连接，因为读操作沿着这些单元传播，就好像它们是组合逻辑，而写操作是由时钟控制的。

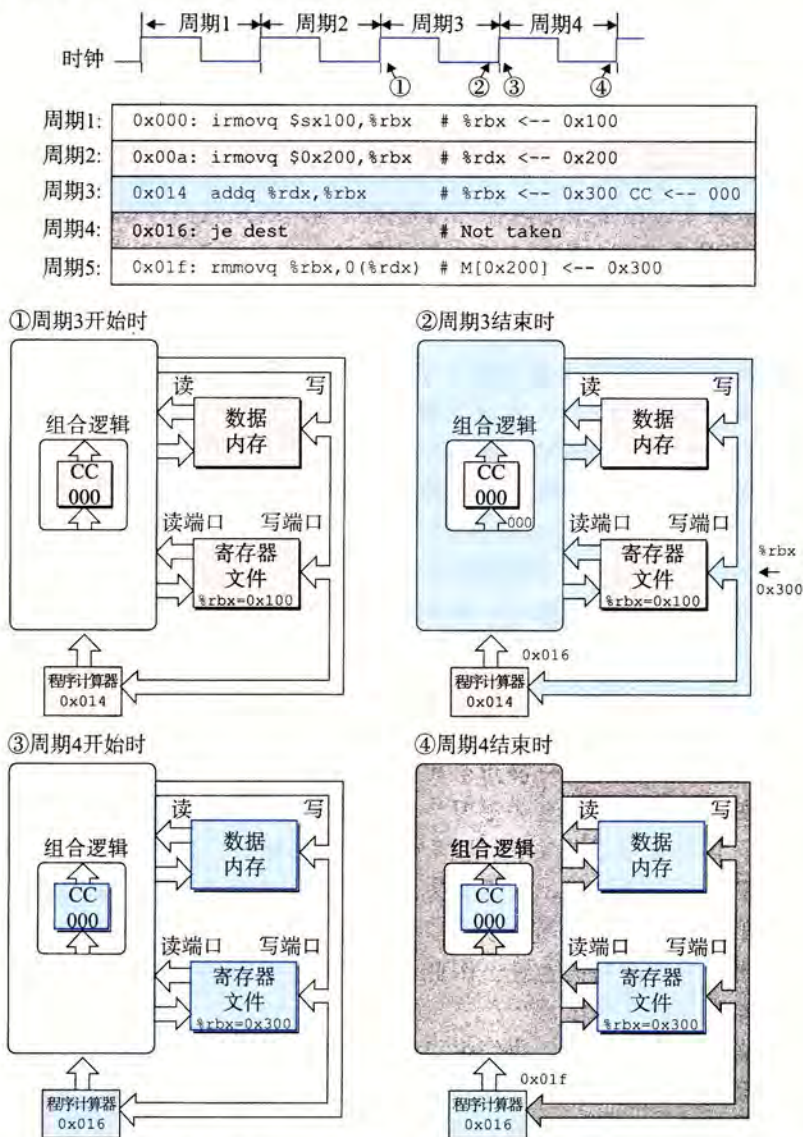


图 4-25 跟踪 SEQ 的两个执行周期。每个周期开始时，状态单元(程序计数器、条件码寄存器、寄存器文件以及数据内存)是根据前一条指令设置的。信号传播通过组合逻辑，创建出新的状态单元的值。在下一个周期开始时，这些值会被加载到状态单元中

图 4-25 中的不同颜色的代码表明电路信号是如何与正在被执行的不同指令相联系的。我们假设处理是从设置条件码开始的，按照 ZF、SF 和 OF 的顺序，设为 100。在时钟周期 3 开始的时候(点 1)，状态单元保持的是第二条 `irmovq` 指令(表中第 2 行)更新过的状态，该指令用浅灰色表示。组合逻辑用白色表示，表明它还没有来得及对变化了的状态做出反应。时钟周期开始时，地址 0x014 载入程序计数器中。这样就会取出和处理 `addq` 指令(表中第 3 行)。值沿着组合逻辑流动，包括读随机访问存储器。在这个周期末尾(点 2)，组合逻辑为条件码产生了新的值(000)，程序寄存器 `%rbx` 的更新值，以及程序计数器的新

值(0x016)。在此时,组合逻辑已经根据 addq 指令被更新了,但是状态还是保持着第二条 irmovq 指令(用浅灰色表示)设置的值。

当时钟上升开始周期 4 时(点 3),会更新程序计数器、寄存器文件和条件码寄存器,因此我们用蓝色来表示,但是组合逻辑还没有对这些变化做出反应,所以用白色表示。在这个周期内,会取出并执行 je 指令(表中第 4 行),在图中用深灰色表示。因为条件码 ZF 为 0,所以不会选择分支。在这个周期末尾(点 4),程序计数器已经产生了新值 0x01f。组合逻辑已经根据 je 指令(用深灰色表示)被更新过了,但是直到下个周期开始之前,状态还是保持着 addq 指令(用蓝色表示)设置的值。

如此例所示,用时钟来控制状态单元的更新,以及值通过组合逻辑来传播,足够控制我们 SEQ 实现中每条指令执行的计算了。每次时钟由低变高时,处理器开始执行一条新指令。

4.3.4 SEQ 阶段的实现

本节会设计实现 SEQ 所需要的控制逻辑块的 HCL 描述。完整的 SEQ 的 HCL 描述请参见网络旁注 ARCH:HCL。在此,我们给出一些例子,而其他的作为练习题。建议你做做这些练习来检验你的理解,即这些块是如何与不同指令的计算需求相联系的。

我们没有讲的那部分 SEQ 的 HCL 描述,是不同整数和布尔信号的定义,它们可以作为 HCL 操作的参数。其中包括不同硬件信号的名字,以及不同指令代码、功能码、寄存器名字、ALU 操作和状态码的常数值。只列出了那些在控制逻辑中必须被显式引用的常数。图 4-26 列出了我们使用的常数。按照习惯,常数值都是大写的。

名称	值(十六进制)	含义
IHALT	0	halt 指令的代码
INOP	1	nop 指令的代码
IRRMVQ	2	rrmovq 指令的代码
IIRMOVQ	3	irmovq 指令的代码
IRMMOVQ	4	rmmovq 指令的代码
IMRMVQ	5	mrmovq 指令的代码
IOPL	6	整数运算指令的代码
IJXX	7	跳转指令的代码
ICALL	8	call 指令的代码
IRET	9	ret 指令的代码
IPUSHQ	A	pushq 指令的代码
IPOPQ	B	popq 指令的代码
FNONE	0	默认功能码
RRSP	4	%rsp 的寄存器 ID
RNONE	F	表明没有寄存器文件访问
ALUADD	0	加法运算的功能
SAOK	1	①正常操作状态码
SADR	2	②地址异常状态码
SINS	3	③非法指令异常状态码
SHLT	4	④halt 状态码

图 4-26 HCL 描述中使用的常数值。这些值表示的是指令、功能码、寄存器 ID、ALU 操作和状态码的编码

除了图 4-18~图 4-21 中所示的指令以外,还包括了对 nop 和 halt 指令的处理。nop

指令只是简单地经过各个阶段，除了要将 PC 加 1，不进行任何处理。halt 指令使得处理器状态被设置为 HLT，导致处理器停止运行。

1. 取指阶段

如图 4-27 所示，取指阶段包括指令内存硬件单元。以 PC 作为第一个字节(字节 0)的地址，这个单元一次从内存读出 10 个字节。第一个字节被解释成指令字节，(标号为“Split”的单元)分为两个 4 位的数。然后，标号为“icode”和“ifun”的控制逻辑块计算指令和功能码，或者使之等于从内存读出的值，或者当指令地址不合法时(由信号 imem_error 指明)，使这些值对应于 nop 指令。根据 icode 的值，我们可以计算三个一位的信号(用虚线表示)：

instr_valid: 这个字节对应于一个合法的 Y86-64 指令吗？这个信号用来发现不合法的指令。

need_regids: 这个指令包括一个寄存器指示符字节吗？

need_valC: 这个指令包括一个常数字吗？

(当指令地址越界时会产生的)信号 instr_valid 和 imem_error 在访存阶段被用来产生状态码。

让我们再来看一个例子，need_regids 的 HCL 描述只是确定了 icode 的值是否为一条带有寄存器指示值字节的指令。

```
bool need_regids =
    icode in { IRRMOVQ, IOPQ, IPUSHQ, IPOPQ,
               IIRMOVQ, IRMMOVQ, IMRMOVQ };
```

 **练习题 4.19** 写出 SEQ 实现中信号 need_valC 的 HCL 代码。

如图 4-27 所示，从指令内存中读出的剩下 9 个字节是寄存器指示符字节和常数字的组合编码。标号为“Align”的硬件单元会处理这些字节，将它们放入寄存器字段和常数字中。当被计算出的信号 need_regids 为 1 时，字节 1 被分开装入寄存器指示符 rA 和 rB 中。否则，这两个字段会被设为 0xF(RNONE)，表明这条指令没有指明寄存器。回想一下(图 4-2)，任何只有一个寄存器操作数的指令，寄存器指示值字节的另一个字段都设为 0xF(RNONE)。因此，可以将信号 rA 和 rB 看成，要么放着我们要访问的寄存器，要么表明不需要访问任何寄存器。这个标号为“Align”的单元还产生常数字 valC。根据信号 need_regids 的值，要么根据字节 1~8 来产生 valC，要么根据字节 2~9 来产生。

PC 增加器硬件单元根据当前的 PC 以及两个信号 need_regids 和 need_valC 的值，产生信号 valP。对于 PC 值 p 、need_regids 值 r 以及 need_valC 值 i ，增加器产生值 $p+1+r+8i$ 。

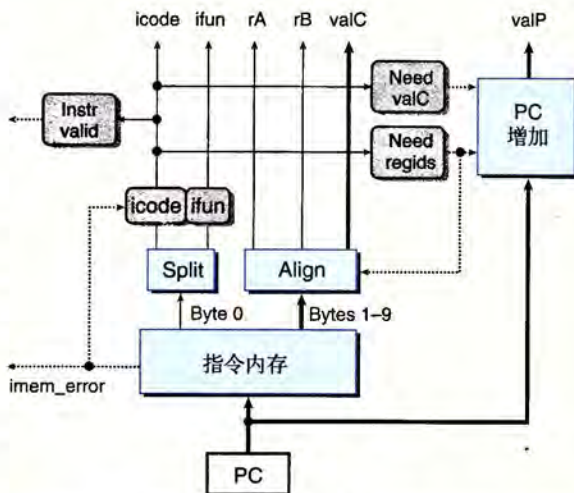


图 4-27 SEQ 的取指阶段。以 PC 作为起始地址，从指令内存中读出 10 个字节。根据这些字节，我们产生出各个指令字段。PC 增加模块计算信号 valP

2. 译码和写回阶段

图 4-28 给出了 SEQ 中实现译码和写回阶段的逻辑的详细情况。把这两个阶段联系在一起是因为它们都要访问寄存器文件。

寄存器文件有四个端口。它支持同时进行两个读(在端口 A 和 B 上)和两个写(在端口 E 和 M 上)。每个端口都有一个地址连接和一个数据连接,地址连接是一个寄存器 ID,而数据连接是一组 64 根线路,既可以作为寄存器文件的输出字(对读端口来说),也可以作为它的输入字(对写端口来说)。两个读端口的地址输入为 srcA 和 srcB,而两个写端口的地址输入为 dstE 和 dstM。如果某个地址端口上的值为特殊标识符 0xF(RNONE),则表明不需要访问寄存器。

根据指令代码 icode 以及寄存器指示值 rA 和 rB,可能还会根据执行阶段计算出的 Cnd 条件信号,图 4-28 底部的四个块产生出四个不同的寄存器文件的寄存器 ID。寄存器 ID srcA 表明应该读哪个寄存器以产生 valA。所需要的值依赖于指令类型,如图 4-18~图 4-21 中译码阶段第一行中所示。将所有这些条目都整合到一个计算中就得到下面的 srcA 的 HCL 描述(回想 RRSP 是 %rsp 的寄存器 ID):

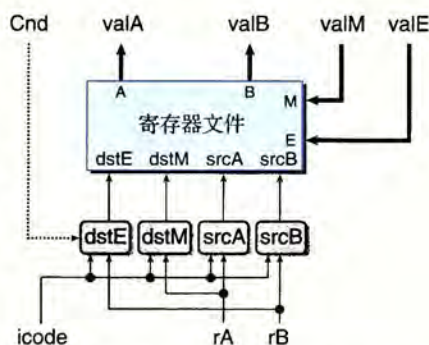


图 4-28 SEQ 的译码和写回阶段。指令字段译码,产生寄存器文件使用的四个地址(两个读和两个写)的寄存器标识符。从寄存器文件中读出的值成为信号 valA 和 valB。两个写回值 valE 和 valM 作为写操作的数据

```
word srcA = [
    icode in { IRRMOVQ, IRMMOVQ, IOPQ, IPUSHQ } : rA;
    icode in { IPOPQ, IRET } : RRSP;
    1 : RNONE; # Don't need register
];
```

练习题 4.20 寄存器信号 srcB 表明应该读哪个寄存器以产生信号 valB。所需要的值如图 4-18~图 4-21 中译码阶段第二步所示。写出 srcB 的 HCL 代码。

寄存器 ID dstE 表明写端口 E 的目的寄存器,计算出来的值 valE 将放在那里。图 4-18~图 4-21 写回阶段第一步表明了这一点。如果我们暂时忽略条件移动指令,综合所有不同指令的目的寄存器,就得到下面的 dstE 的 HCL 描述:

```
# WARNING: Conditional move not implemented correctly here
word dstE = [
    icode in { IRRMOVQ } : rB;
    icode in { IIRMOVQ, IOPQ } : rB;
    icode in { IPUSHQ, IPOPQ, ICALL, IRET } : RRSP;
    1 : RNONE; # Don't write any register
];
```

我们查看执行阶段时,会重新审视这个信号,看看如何实现条件传送。

练习题 4.21 寄存器 ID dstM 表明写端口 M 的目的寄存器,从内存中读出来的值 valM 将放在那里,如图 4-18~图 4-21 中写回阶段第二步所示。写出 dstM 的 HCL 代码。

练习题 4.22 只有 popq 指令会同时用到寄存器文件的两个写端口。对于指令 popq

%rsp, E 和 M 两个写端口会用到同一个地址, 但是写入的数据不同。为了解决这个冲突, 必须对两个写端口设立一个优先级, 这样一来, 当同一个周期内两个写端口都试图对一个寄存器进行写时, 只有较高优先级端口上的写才会发生。那么要实现练习题 4.8 中确定的行为, 哪个端口该具有较高的优先级呢?

3. 执行阶段

执行阶段包括算术/逻辑单元(ALU)。这个单元根据 alufun 信号的设置, 对输入 aluA 和 aluB 执行 ADD、SUBTRACT、AND 或 EXCLUSIVE-OR 运算。如图 4-29 所示, 这些数据和控制信号是由三个控制块产生的。ALU 的输出就是 valE 信号。

在图 4-18~图 4-21 中, 执行阶段的第一步就是每条指令的 ALU 计算。列出的操作数 aluB 在前面, 后面是 aluA, 这样是为了保证 subq 指令是 valB 减去 valA。可以看到, 根据指令的类型, aluA 的值可以是 valA、valC, 或者是一8 或 +8。因此我们可以用下面的方式来表达产生 aluA 的控制块的行为:

```
word aluA = [
    icode in { IRRMOVQ, IOPQ } : valA;
    icode in { IIRMOVQ, IRMMOVQ, IMRMOVQ } : valC;
    icode in { ICALL, IPUSHQ } : -8;
    icode in { IRET, IPOPQ } : 8;
    # Other instructions don't need ALU
];
```

 **练习题 4.23** 根据图 4-18~图 4-21 中执行阶段第一步的第一个操作数, 写出 SEQ 中信号 aluB 的 HCL 描述。

观察 ALU 在执行阶段执行的操作, 可以看到它通常作为加法器来使用。不过, 对于 OPq 指令, 我们希望它使用指令 ifun 字段中编码的操作。因此, 可以将 ALU 控制的 HCL 描述写成:

```
word alufun = [
    icode == IOPQ : ifun;
    1 : ALUADD;
];
```

执行阶段还包括条件码寄存器。每次运行时, ALU 都会产生三个与条件码相关的信号——零、符号和溢出。不过, 我们只希望在执行 OPq 指令时才设置条件码。因此产生了一个信号 set_cc 来控制是否该更新条件码寄存器:

```
bool set_cc = icode in { IOPQ };
```

标号为“cond”的硬件单元会根据条件码和功能码来确定是否进行条件分支或者条件数据传送(图 4-3)。它产生信号 Cnd, 用于设置条件传送的 dstE, 也用在条件分支的下一个 PC 逻辑中。对于其他指令, 取决于指令的功能码和条件码的设置, Cnd 信号可以被设置为 1 或者 0。但是控制逻辑会忽略它。我们省略这个单元的详细设计。

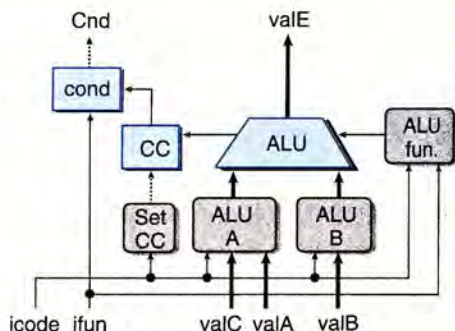


图 4-29 SEQ 执行阶段。ALU 要么为整数运算指令执行操作, 要么作为加法器。根据 ALU 的值, 设置条件码寄存器。检测条件码的值, 判断是否该选择分支

练习题 4.24 条件传送指令(简称 `cmovXX`)的指令代码为 `IRRMovQ`。如图 4-28 所示,我们可以用执行阶段中产生的 `Cnd` 信号实现这些指令。修改 `dstE` 的 HCL 代码以实现这些指令。

4. 访存阶段

访存阶段的任务就是读或者写程序数据。如图 4-30 所示,两个控制块产生内存地址和内存输入数据(为写操作)的值。另外两个块产生表明应该执行读操作还是写操作的控制信号。当执行读操作时,数据内存产生值 `valM`。

图 4-18~图 4-21 的访存阶段给出了每个指令类型所需要的内存操作。可以看到内存读和写的地址总是 `valE` 或 `valA`。这个块用 HCL 描述就是:

```
word mem_addr = [
    icode in { IRMMOVQ, IPUSHQ, ICALL, IMRMovQ } : valE;
    icode in { IPOPQ, IRET } : valA;
    # Other instructions don't need address
];
```

练习题 4.25 观察图 4-18~图 4-21 所示的不同指令的访存操作,我们可以看到内存写的地址总是 `valA` 或 `valP`。写出 SEQ 中信号 `mem_data` 的 HCL 代码。

我们希望只为从内存读数据的指令设置控制信号 `mem_read`, 用 HCL 代码表示就是:

```
bool mem_read = icode in { IMRMovQ, IPOPQ, IRET };
```

练习题 4.26 我们希望只为向内存写数据的指令设置控制信号 `mem_write`。写出 SEQ 中信号 `mem_write` 的 HCL 代码。

访存阶段最后的功能是根据取值阶段产生的 `icode`、`imem_error`、`instr_valid` 值以及数据内存产生的 `dmem_error` 信号,从指令执行的结果来计算状态码 `Stat`。

练习题 4.27 写出 `Stat` 的 HCL 代码,产生四个状态码 `SAOK`、`SADR`、`SINS` 和 `SHLT`(参见图 4-26)。

5. 更新 PC 阶段

SEQ 中最后一个阶段会产生程序计数器的新值(见图 4-31)。如图 4-18~图 4-21 中最后步骤所示,依据指令的类型和是否要选择分支,新的 PC 可能是 `valC`、`valM` 或 `valP`。用 HCL 来描述这个选择就是:

```
word new_pc = [
    # Call. Use instruction constant
    icode == ICALL : valC;
    # Taken branch. Use instruction constant
    icode == IJXX && Cnd : valC;
    # Completion of RET instruction. Use value from stack
```

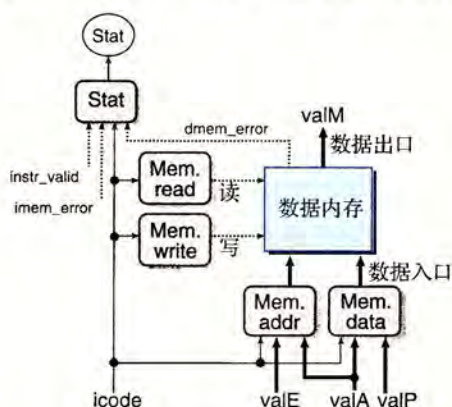


图 4-30 SEQ 访存阶段。数据内存既可以写,也可以读内存的值。从内存中读出的值就形成了信号 `valM`

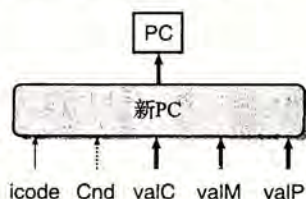


图 4-31 SEQ 更新 PC 阶段。根据指令代码和分支标志,从信号 `valC`、`valM` 和 `valP` 中选出下一个 PC 的值


```

        icode == IRET : valM;
        # Default: Use incremented PC
        1 : valP;
    ];

```

6. SEQ 小结

现在我们已经浏览了 Y86-64 处理器的一个完整的设计。可以看到，通过将执行每条不同指令所需的步骤组织成一个统一的流程，就可以用很少量的各种硬件单元以及一个时钟来控制计算的顺序，从而实现整个处理器。不过这样一来，控制逻辑就必须要在这些单元之间路由信号，并根据指令类型和分支条件产生适当的控制信号。

SEQ 唯一的问题就是它太慢了。时钟必须非常慢，以使信号能在一个周期内传播所有的阶段。让我们来看看处理一条 `ret` 指令的例子。在时钟周期起始时，从更新过的 PC 开始，要从指令内存中读出指令，从寄存器文件中读出栈指针，ALU 将栈指针加 8，为了得到程序计数器的下一个值，还要从内存中读出返回地址。所有这一切都必须在这个周期结束之前完成。

这种实现方法不能充分利用硬件单元，因为每个单元只在整个时钟周期的一部分时间内才被使用。我们会看到引入流水线能获得更好的性能。

4.4 流水线的通用原理

在试图设计一个流水线化的 Y86-64 处理器之前，让我们先来看看流水线化的系统的一些通用属性和原理。对于曾经在自助餐厅的服务线上工作过或者开车通过自动汽车清洗线的人，都会非常熟悉这种系统。在流水线化的系统中，待执行的任务被划分成了若干个独立的阶段。在自助餐厅，这些阶段包括提供沙拉、主菜、甜点以及饮料。在汽车清洗中，这些阶段包括喷水和打肥皂、擦洗、上蜡和烘干。通常都会允许多个顾客同时经过系统，而不是要等到一个用户完成了所有从头至尾的过程才让下一个开始。在一个典型的自助餐厅流水线上，顾客按照相同的顺序经过各个阶段，即使他们并不需要某些菜。在汽车清洗的情况中，当前面一辆汽车从喷水阶段进入擦洗阶段时，下一辆就可以进入喷水阶段了。通常，汽车必须以相同的速度通过这个系统，避免撞车。

流水线化的一个重要特性就是提高了系统的吞吐量(throughput)，也就是单位时间内服务的顾客总数，不过它也会轻微地增加延迟(latency)，也就是服务一个用户所需要的时间。例如，自助餐厅里的一个只需要甜点的顾客，能很快通过一个非流水线化的系统，只在甜点阶段停留。但是在流水线化的系统中，这个顾客如果试图直接去甜点阶段就有可能招致其他顾客的愤怒了。

4.4.1 计算流水线

让我们把注意力放到计算流水线上来，这里的“顾客”就是指令，每个阶段完成指令执行的一部分。图 4-32a 给出了一个很简单的非流水线化的硬件系统例子。它是由一些执行计算的逻辑以及一个保存计算结果的寄存器组成的。时钟信号控制在每个特定的时间间隔加载寄存器。CD 播放器中的译码器就是这样的一个系统。输入信号是从 CD 表面读出的位，逻辑电路对这些位进行译码，产生音频信号。图中的计算块是用组合逻辑来实现的，意味着信号会穿过一系列逻辑门，在一定时间的延迟之后，输出就成为了输入的某个函数。